

1 Problemes Tractables vs Problemes Intractables

1. **2-color vs 3-color.** Recordeu que un graf no dirigit $G = (V, E)$ és *bipartit* si existeix una partició de V en V_1, V_2 de manera que tota aresta $uv \in E$ satisfà o bé $u \in V_1 \wedge u \in V_2$ o bé $u \in V_2 \wedge u \in V_1$. Fixeu-vos que dir que un graf és bipartit és equivalent a dir que és 2-colorable (2-COLOR). Definim $\text{BIPARTITE GRAPH} = \{\langle G \rangle \mid G \text{ és bipartit}\}$.

Demostreu que $\text{BIPARTITE GRAPH} \in \text{P}$.

JA FETS ALTRE

PDF

2. **DNF-Sat vs CNF-Sat.** Demostreu que el problemes següents pertanyen a la classe P:

- (a) Donada una fórmula booleana en *Forma Normal Conjuntiva (CNF)* on cada clàusula té com a màxim dos literals, decidir si és satisfactible.
- (b) Donada una fórmula booleana en *Forma Normal Disjuntiva (DNF)*, decidir si és satisfactible.
- (c) Una fórmula booleana en CNF es pot transformar en una fórmula booleana en DNF equivalent? Si es pot, en quant temps?

3. **Eulerian graph vs Hamiltonian graph** Recordeu que un *recorregut eulerià* en un graf no dirigit és un cicle que pot visitar repetides vegades un mateix vèrtex i ha d'utilitzar cada aresta exactament una vegada. Diem que un graf és *eulerià* si conté un recorregut eulerià.

Demostreu que

$\text{EULERIAN GRAPH} = \{\langle G \rangle \mid G \text{ és eulerià}\} \in \text{P}.$

Dissenyeu un algorisme eficient que en el cas que el graf sigui eulerià ens retorni un recorregut eulerià.

4. **Shortest path vs longest path** Considerem les dues versions següents de *Shortest Path* i *Longest Path*, respectivament, per a grafs no dirigits:

$\text{SPATH} = \{\langle G, a, b, k \rangle \mid G \text{ conté un camí simple de } a \text{ a } b \text{ de longitud menor o igual que } k\}$

$\text{LPATH} = \{\langle G, a, b, k \rangle \mid G \text{ conté un camí simple de } a \text{ a } b \text{ de longitud més gran o igual que } k\}$.

- (a) Demostreu que $\text{SPATH} \in \text{P}$.
- (b) Demostreu que $\text{LPATH} \in \text{NP}$. Demostreu que si $\text{LPATH} \in \text{P}$ aleshores el problema del camí Hamiltonià en graf no dirigits també seria a P .

5. Consider the problem MODULAR EXPONENTIATION: given $a, b, n \in \mathbb{N}$, compute $d = a^b \bmod n$.
Can MODULAR EXPONENTIATION be solved in polynomial time?

6. **Half clique.** Considereu el problema SUBGRAF COMPLET D'ORDRE MEITAT (HALF-CLIQUE):

Donat un graf no dirigit G , decidir si conté una $\lfloor \frac{n}{2} \rfloor$ -clique.

(a) Demostreu que HALF-CLIQUE $\in$ NP

(b) Demostreu que si HALF-CLIQUE $\in$ P, aleshores CLIQUE $\in$ P.

7. **Self reducibility SAT.** Supposem que tenim un algorisme que decideix SAT en temps polinòmic. Dissenyeu un algorisme tal que donada una fórmula boolena F retorni una assignació que la satisfaci, si existeix. Demostreu que si $\text{SAT} \in \text{P}$ aleshores *trobar* una assignació que la satisfà també és computable en temps polinòmic. Aquesta propietat s'anomena *self-reducibility*.

8. **Self reducibility CLIQUE.** Supposem que tenim un algorisme que decideix CLIQUE en temps polinòmic. Dissenyeu un algorisme tal que donat un graf G i donat un natural k , retorni una clica de k vèrtexs, si existeix. Demostreu que si CLIQUE $\in$ P aleshores *trobar* una clique també és computable en temps polinòmic. Aquesta propietat s'anomena *self-reducibility*.

9. **Stingy SAT.** Let us consider the following problem:

STINGY SAT Given a set of clauses (each a disjunction of literals) and an integer k , find a satisfying assignment in which at most k variables are true, if such an assignment exists.

Prove that STINGY SAT is NP-complete.

10. **At most 3.** Consider the CLIQUE problem restricted to graphs in which every vertex has degree at most 3. Call this problem CLIQUE-3.

- (a) Prove that CLIQUE-3 is in NP.
- (b) What is wrong with the following proof of NP-completeness for CLIQUE-3? We know that the CLIQUE problem in general graphs is NP-complete, so it is enough to present a reduction from CLIQUE-3 to CLIQUE. Given a graph G with vertices of degree ≤ 3 , and a parameter k , the reduction leaves the graph and parameter unchanged: clearly the output for the reduction is a possible input for the CLIQUE problem. Furthermore, the answer to both problems is identical. This proves the correctness of the reductions, and, therefore, the NP-completeness of CLIQUE-3.
- (c) It is true that the VERTEX COVER problem remains NP-complete even when restricted to graphs in which every vertex has degree at most 3. Call this problem VC-3. What is wrong in the following proof of NP-completeness for CLIQUE-3?
We present a reduction from VC-3 to CLIQUE-3. Given a graph $G = (V, E)$ with node degrees bounded by 3, and a parameter b , we create an instance of CLIQUE-3 by leaving the graph unchanged and switching the parameter to $|V| - b$. Now, a subset $C \subseteq V$ is a vertex cover in G if and only if the complementary set $V - C$ is a clique in G . Therefore G has a vertex cover of size $\leq b$ if and only if it has a clique of size $\geq |V| - b$. This proves the correctness of the reduction and, consequently, the NP-completeness of CLIQUE-3.
- (d) Describe an $O(|V|)$ algorithm for CLIQUE-3.

11. **At most twice.** Show that 3SAT remains NP-complete even when restricted to formulas in which each literal appears at most twice.

$$x \vee \neg x$$

Original 3SAT_{≤2}

$$3SAT \leq_m^{\uparrow} 3SAT_{\leq 2}$$

- Testimoni matrix a SAT
- Verifier matrix a SAT

$$f(x) \in \text{in } A \leq 6$$

input F

$$\forall x \in F$$

$$\text{app}[x] \text{ ++ (operations)}$$

$$\text{si } \forall l \in F \text{ app} \leq 2$$

output F

Since

$$\text{sea } k \leftarrow \text{app}[x]$$

generamos $a_1, \dots, a_k$

i los clausulas remant $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_k \rightarrow a_1$

ex:

$$\left[\begin{array}{cccc} x_1 & \neg x_1 & x_1 & x_1 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ a & b & c & d \\ a \rightarrow b \rightarrow c \rightarrow d \rightarrow x_1 \rightarrow a \end{array} \right]$$

per cada $a_i \rightarrow a_{i+1}$ ofegim clàusula $\neg a_i \vee a_{i+1}$
 substituir

$$\underline{F_{\text{sat}} \Rightarrow F'_{\text{sat}}}$$

$$\alpha(x_i) \rightarrow \alpha(x'_i) = \alpha(x_i)$$

$$\alpha(\text{lim } x'_i) = \alpha(x_i)$$

$$\underline{F_{\text{sat}} \Leftarrow F'_{\text{sat}}}$$

$$\alpha(x_i) \leftarrow \begin{matrix} \exists \alpha(x_i) \\ \alpha(\text{lim } x'_i) \end{matrix}$$

12. **Degree at most 4** In the previous exercise we saw that 3SAT remains NP-complete even when restricted to formulas in which each literal appears at most twice.

- (a) Show that if each literal appears at most once, then the problem is solvable in polynomial time.
- (b) Show that INDEPENDENT SET remains NP-complete even in the special case when all nodes in the graph have degree at most 4.

a) Podem tenir com a molt x_i i $-x_i$ (per literal diferent)

La fórmula serà del tipus: $F = (x_1 \vee -x_1 \vee x_2) \wedge (-x_2 \vee x_3 \vee x_4) \wedge (-x_4 \vee x_5 \vee -x_3)$

Per el teorema de Hall, podem agafar sempre una variable de cada clàusula (+ o -) i li assignem true o false i, d'aquesta forma sempre tindrem que F és satisfactible.

$O(1)$ és 3SAT F ? Sí, sempre

$O(\sqrt{V})$ per la nostra assignació, algoritme Hopcroft-Karp.

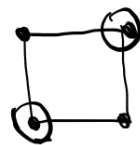
b) Independent Set

Subconjunt $S \subseteq V$ on cap vèrtex $v \in S$ és adjacent amb altre, és a dir:

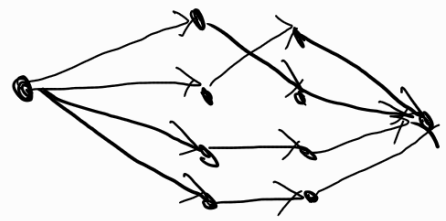
$$(\forall u, v \in S: (u, v) \notin E)$$

(Independent Set $\iff$ és Clique al complementari)

Per veure si IS-4 és NP: $[3SAT \leq_m IS-4]$
complet pt most twice



Done



a) Dinic (ulgarit - fluxa) / Pref match o fluxa)

$(F, cond)$



$$(C_i, x_{ij}) \iff x_{ij} \in C_i$$



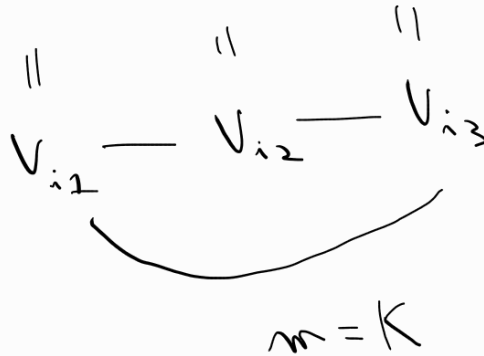
$$\neg x_{ij} \in C_i$$



($\exists$ for $R_2(F)$ find an element $a[]$
Since term $[]$ unique variable
for for double sets

b)

$$C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$$



$$\text{var } x: x \neg x$$

(convertion)
and nega)

$$F = C_1 \wedge \dots \wedge C_K$$

13. Consider a set of linear inequalities where all the coefficients are integers. Consider the problem INTLINEQ of deciding if a given set of linear inequalities has an integer solution. Related to this problem we can consider a variant of linear programming INTEGER PROGRAMMING in which the variables are restricted to be integers.

Show that INTLINEQ is NP-complete and that INTEGER PROGRAMMING is NP-hard.

func obj o var

Entonces: matrix

Int Line Eq $\in$ NP

$\left\{ \begin{array}{l} \text{Certificat} = (Vx, Vx_1, \dots, Vx_m) \\ \text{Verificador} = \text{miron ecuació per ecuació} \end{array} \right.$

$m \text{ eq}$ $n \text{ var}$
 $\uparrow$ $\uparrow$
 $(m \cdot n)$

$3SAT \leq_m^{\uparrow} \text{Int Line Eq}$

$C_i = \{x_1, x_2, \neg x_3\}$

$$1x_1 + 1x_2 + (1 - x_3) \geq 1$$

→ same func obj o maximization

Integer Programming $\in$ NP-HARD

14. Lucas' theorem says the following: If we have an integer a such that: $a^{n-1} \equiv 1 \pmod{n}$, and, for every prime factor q of $n-1$, it is not the case that $a^{(n-1)/q} \equiv 1 \pmod{n}$, then n is prime.

Can this result be used to show that Primality belongs to NP?

• Si, s'anomena el certificat de Pratt:

Necesitem un verificador polinomial i un testimoni per demostrar que Primality $\in$ NP. La entrada n la podem expressar com a $k = \lceil \log_2(n) \rceil$ bits.

El teorema de Lucas ens dona les condicions matemàtiques. El verificador rep com a testimoni ' a ' i els divisors primers ' $q_1, q_2, \dots, q_m$ ' de $n-1$. El verificador ha de comprovar que tots els q_i són primers! Per tant el [testimoni] ha de ser recursiu, i ha de contenir:

- nombre a
- tots els q_i i els seus exponents
- un testimoni de primalitat per tot q_i

Mida testimoni

Un nombre com a molt té $\log_2 n$ factors primers. Al primer nivell de recursió tenim com a molt $\log_2 n$ factors. En el següent nivell factoritzem aquests factors, i així successivament.

Com a cada nivell almenys dividim per 2, la profunditat de recursió és $O(\log_2 n)$. Com cada node ocupa com a màxim $\log_2 n$ bits, la mida total és $\log_2(n) \cdot \log_2(n) = O(\log_2^2 n)$.

Verificador polinòmic

El que ho de fer és comprovar que $g_1^{e_1} \cdot g_2^{e_2} \cdot \dots \cdot g_m^{e_m} = n-1$, i comprovar que que $(a^{n-1} \equiv 1 \pmod n)$ i que $(a^{\frac{n-1}{4}} \neq 1)$.
Lo qual tota és trivialment polinòmica.

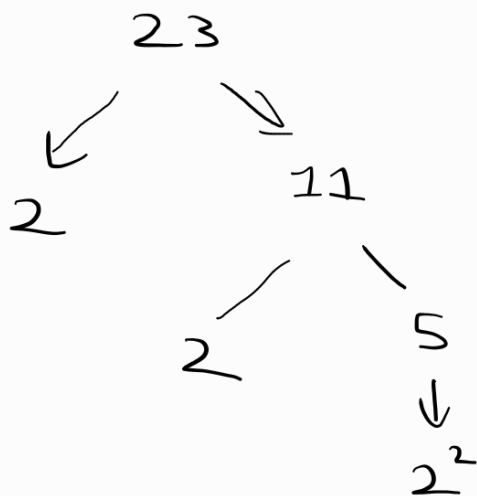
Exemple altre recursiu i amb $n=23$

1 - calculem $n-1 = 22$

2 - la factorització és 11×2

3 - Per demostrar teorema de Wilson necessitem un testimoni $a=5$ (p.e.a) i demostrar 11 y 2 són primers

...



15. Consider the following problems:

- MODULAR FACTORIAL: Given N bits, natural numbers x, y , compute $x! \bmod y$.
- SMALLEST PRIME DIVISOR: Given a N bit natural number x , compute the smallest prime divisor of x .
- FACTORING: Given a N bit natural number x , compute the factorization of x as product of primes.

(a) Prove that y is prime if and only if, for each integer $x < y$, we have that $\gcd(x!, y) = 1$.

(b) Show that if MODULAR FACTORIAL can be solved in polynomial time, then SMALLEST PRIME DIVISOR and FACTORING could be solved in polynomial time.

$\gcd = \text{màxim divisor comú}$

a) Ens donem $\gcd(x!, y) = 1 \Rightarrow y$ é primer

Demostració per R.A. $P \Rightarrow Q \equiv P \Rightarrow \neg Q$ i anirem a contradi.

Suposem que y no é primer, llavors està format per dos nombres a i b tals que $y = a \cdot b$, $1 < a, b < y$.

Si escollim $x = a$, es compleix $x < y$. El factorial de x seria $a \cdot (a-1) \cdot (a-2) \dots \cdot 1$. Per la tant $\gcd(x!, y) \geq a > 1$ y anirem a una contradicció!!

b) Anuntat Modular Factorial é polinòmic venem:

• SMALLEST PRIME DIVISOR é polinòmic

Amh l'opartat anterior: $\gcd(x!, N) > 1 \Leftrightarrow N$ té un factor $\leq x$.

Farem l'úniquesa binària per a trobar aquest valor! $O(\log n)$.

• Si 1 hem de pujar (dreta)

• Si > 1 intentem baixar (esquerra)

7, 1, 1, 1, ..., >1, >1, >1
 $\uparrow$
 trobem aquest

• FACTORIZAÇÃO

Usam o anterior para achar o divisor primo mais petit de N .
Sejam $N' = \frac{N}{p}$, e seguimos assim até chegar a 1. $\square$